A Leakage-Aware Benchmark for Ultra-Small Language Models on Structured Speech-Command Routing

[Removed for double-blind review]

[1][Removed for double-blind review]

Abstract. *Motivated by edge and on-device deployment scenarios, structured command routing benefits from ultra-small language models ($\leq$ 3B parameters). We present a reproducible benchmark of five sub-3B instruction-tuned models across four families on a transcript-to-JSON routing task derived from Fluent Speech Commands (FSC). We evaluate Qwen2.5-0.5B, Qwen3.5-0.8B, TinyLlama-1.1B, SmolLM2-1.7B, and Qwen3.5-2B under a canonical two-tool contract with policy-defined abstention, evaluating both official speaker-disjoint ($n = 1{,}934$) and audited phrase-disjoint partitions ($n = 2{,}296$, 38 template clusters). On phrase-disjoint test, a transparent lexical baseline achieves 77.40% exact match, outperforming every zero-shot neural model. Qwen3.5-2B attains 100% contract validity but collapses to universal abstention on phrase-disjoint test (0% exact call recall). Three models achieve 0% contract validity due to unconstrained schema deviations, while Qwen3.5-0.8B reaches 60.15% validity and 43.60% exact match. A supervised LoRA adapter provides in-distribution proof of concept. Our audit shows that syntactic compliance, policy abstention, and semantic accuracy must be decoupled when evaluating edge SLMs.*

Keywords: ultra-small language models; structured prediction; speech commands; function routing; abstention; data leakage; reproducible benchmarking.

1. Scope and Research Questions

Motivated by edge and on-device deployment scenarios, evaluating local language processing on resource-constrained hardware has driven the development of ultra-small language models (SLMs, $\leq$ 3B parameters) [5, 6]. While function calling and tool execution are established in large cloud models [13, 14], evaluating whether sub-3B instruction-tuned models can reliably route commands under strict output schemas remains an open question. This article deliberately narrows the mobile-assistant setting to a benchmark of ultra-small language models. The parameter cap is strict: no checkpoint with more than 3,000,000,000 parameters is included (evaluated models range from 0.494B to 2.274B). The input is an English human speech transcription, and the output is a validated JSON object. The benchmark does not claim Brazilian-Portuguese coverage, Android execution, automatic speech recognition (ASR) quality, battery behavior, thermal behavior, or smartphone latency. The physical phone is not required.

We ask: (i) how do checkpoints below the parameter cap differ in JSON compliance and canonical command accuracy under one declared contract? (ii) how much does exact-match performance change when normalized transcription templates are held out,

rather than only speakers? (iii) do simple controls provide a meaningful reference for the models? and (iv) how different are item-level and template-cluster summaries when repeated phrasings occur across speakers?

2. Human Corpus and Derived Task

We use Fluent Speech Commands (FSC), introduced as a human-speech corpus for end-to-end spoken-language understanding [1], contrasting with keyword-spotting corpora and commercial edge SLU datasets like SNIPS [4]. It contains 30,043 human audio recordings from 97 speakers across 31 native intent categories, with official train, validation, and test partitions. We use the human-origin transcriptions and split metadata, not the raw waveforms. The raw archive remains outside Git and is identified by SHA-256 in the server-local manifest.

FSC is an intent-classification corpus, not a native function-calling dataset. We therefore define an explicit, generic two-operation contract. Only the following four native combinations become calls:

Table 1. The four deterministic FSC-to-contract bridge rules.

Native action	Native object	Derived target
activate	music	`media_control(action=play)`
deactivate	music	`media_control(action=pause)`
increase	volume	`volume_adjust(direction=up)`
decrease	volume	`volume_adjust(direction=down)`

Every other retained utterance receives a canonical policy abstention target. The output must contain exactly three required root keys: `action`, `tool`, and `arguments`, with additional fields forbidden (`additionalProperties: false`). The top-level `action` is either `call` or `abstain`. Valid calls specify `media_control` with argument `action` in {`play`, `pause`}, or `volume_adjust` with argument `direction` in {`up`, `down`}. Policy abstention is represented strictly as {`"action": "abstain", "tool": null, "arguments": {}`}. Evaluation parses candidate outputs as JSON and evaluates semantic object equality against the derived gold object, making evaluation invariant to key serialization order and whitespace formatting.

To prevent unsupported intents from dominating the evaluation, the dataset generator samples a balanced 1:1 mixture of calls and policy abstentions within each split. Across all three partitions (train, dev, and test), each derived dataset contains 14,956 records: 7,478 calls and 7,478 policy abstentions. The official speaker-disjoint test split ($n = 1{,}934$) contains 967 calls and 967 abstentions, derived from FSC's 3,793 test audio recordings. The phrase-disjoint test split ($n = 2{,}296$) contains 1,148 calls and 1,148 abstentions. The contract serves strictly as an evaluation target and does not execute tools or grant system permissions.

3. Leakage-Audited Protocols

The first protocol preserves FSC's official speaker-disjoint split, measuring transfer to held-out speakers without preventing identical phrasings across partitions. The second

protocol partitions normalized transcriptions to ensure zero template overlap across splits. Each transcription is normalized using Unicode NFKC, case-folding, whitespace collapsing, and punctuation removal except apostrophes. For each native FSC intent tuple (`action`, `object`, `location`) across all 31 intents, unique normalized template digests (`SHA-256(f"{seed}|{key}|{template_id}")`, seed 20260830) are assigned to target train/dev/test proportions of approximately 70/15/15 using deterministic rounding. Speakers may occur in more than one split in this second protocol by design.

Table 2. Balanced derived datasets and leakage controls.

Protocol	Train	Dev	Test	Total	Control
Official speaker-disjoint	11,442	1,580	1,934	14,956	speakers
Phrase-disjoint	10,458	2,202	2,296	14,956	templates

The official protocol has zero speaker intersections but 245, 247, and 244 template intersections for train–dev, train–test, and dev–test, respectively. The phrase-disjoint protocol has zero template intersections. It contains 38 test template clusters (9 call and 29 abstain clusters), so cluster confidence intervals are broad; this reflects the empirical sample of unseen phrasing templates in FSC.

4. Models and Execution

The matrix evaluates five open SLM checkpoints: Qwen2.5/3.5 [9], TinyLlama [7], and SmolLM2 [8]. Model parameter counts are computed by summing tensor shapes in local safetensors files, ensuring an auditable cap that includes all weights. While Qwen3.5 checkpoints are exposed as multimodal architectures by Transformers, no visual input is supplied in this speech-transcript experiment.

Table 3. Checkpoints included in the strict ultra-small matrix.

Checkpoint	Parameters	Family	Revision
Qwen2.5-0.5B-Instruct	494,032,768	Qwen2	`7ae5576`
Qwen3.5-0.8B	873,438,784	Qwen3.5	`2fc0636`
TinyLlama-1.1B-Chat	1,100,048,384	Llama	`fe8a4ea`
SmolLM2-1.7B-Instruct	1,711,376,384	SmolLM	`31b70e2`
Qwen3.5-2B	2,274,069,824	Qwen3.5	`15852e8`

The excluded Qwen2.5-3B-Instruct checkpoint has 3,085,938,688 measured parameters and therefore exceeds the strict cap. Qwen3.5-4B-Base is also excluded. Complete IDs, revisions, licenses, and counts are recorded in `benchmarks/ultrasmall_models.json`.

All models use their native chat template with the same English semantic instruction, compact catalog, greedy decoding (`do_sample=false`), and a 64-token generation cap. The template mode and model class are recorded per prediction. Runs use Transformers 5.3.0, PyTorch 2.10.0+cu128, and one NVIDIA GeForce RTX 5090 with 32 GB VRAM. No fine-tuning is part of the main zero-shot matrix. As an in-distribution reference, a supervised Qwen2.5-0.5B LoRA adaptor ($r = 16, \alpha = 32$, dropout 0.05, AdamW lr 2×10^{-4}, effective batch 16, 2 epochs on the official training split) is evaluated on the official test split ($n = 1{,}934$) as a proof of concept.

5. Metrics and Controls

The evaluator distinguishes parseability and JSON validity; contract validity, including action/tool/argument constraints; exact match against the derived target; action accuracy; tool and argument accuracy on call gold items; and abstention precision, recall, and F1.

The primary metric is item-level exact match accompanied by cluster-level summaries. For each template cluster, the evaluator computes mean exact-match rate, action accuracy, abstention F1, and strict cluster match (clusters where 100% of items are correct). Cluster-bootstrap 95% confidence intervals resample template clusters rather than individual rows. Standard Wilson intervals are reported as descriptive row-level statistics but do not capture cluster dependency.

Two deterministic non-neural controls provide baseline references under the contract: `always_abstain` returns policy rejection for every item; `lexical` matches domain keywords (*play, pause, volume, up, down*) without accessing dataset metadata.

6. Results

6.1. Main Phrase-Disjoint Comparison

Table 4. Phrase-disjoint test results (%, $n = 2{,}296$, 38 template clusters). R_{call}^{exact}: exact call recall; "–" indicates undefined F_1 due to zero predicted abstentions.

System	Parse	Valid	Exact	R_{call}^{exact}	Abstain F1	Template exact
Always abstain	100.00	100.00	50.00	0.00	66.67	76.32
Lexical control	100.00	100.00	**77.40**	**54.79**	81.56	**89.47**
Qwen2.5-0.5B	70.17	0.00	0.00	0.00	–	0.00
Qwen3.5-0.8B	100.00	60.15	43.60	33.45	69.92	50.00
TinyLlama-1.1B	43.42	0.00	0.00	0.00	–	0.00
SmolLM2-1.7B	96.47	0.00	0.00	0.00	–	0.00
Qwen3.5-2B	100.00	100.00	50.00	0.00	66.67	76.32

The lexical control outperforms every zero-shot model on exact match. On phrase-disjoint test, Qwen3.5-2B obtains 100% contract validity and 50.00% exact match: its output behavior collapses universally to abstention on 100% of test instances ($R_{call}^{exact} = 0\%$). On official test, Qwen3.5-2B emits 19 correct tool calls ($R_{call}^{exact} = 1.96\%$), reaching 50.98% exact match and 76.11% Template Exact. Qwen3.5-0.8B is the only model with substantial call coverage, yet its exact match remains below the lexical control. The other three checkpoints produce parseable text often enough to be measurable, but their responses do not satisfy the declared canonical schema.

6.2. Official Speaker-Disjoint Comparison

The official split gives Qwen3.5-0.8B a higher exact match than phrase-disjoint (54.65% vs. 43.60%), an observed gap of 11.05 pp when template leakage is eliminated. We treat this divergence as an observational comparison rather than a formal causal test: the template-cluster bootstrap 95% confidence interval for phrase-disjoint exact match is wide ($[26.02\%, 63.35\%]$), reflecting the sample size of 38 held-out clusters (9 call-only and 29

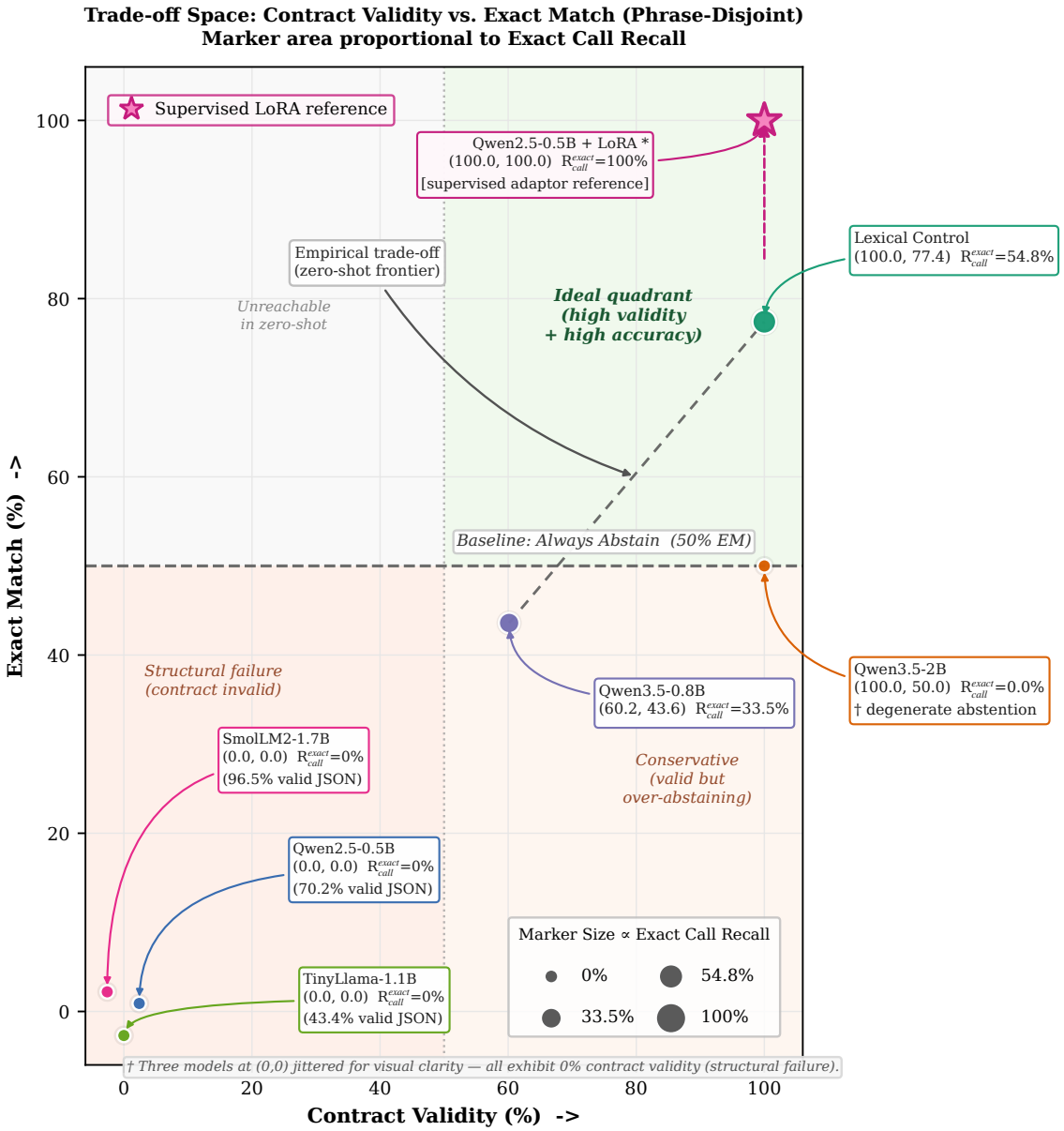

Figure 1. Trade-off space on the phrase-disjoint test ($n = 2{,}296$ items, 38 template clusters). x: contract validity (%), y: exact match (item-level %). Marker area $\propto$ exact call recall (proportion of gold calls with correct tool and arguments). Qwen3.5-2B attains 100% validity via degenerate abstention on phrase-disjoint test (exact call recall 0%); lexical control (100.0, 77.4) dominates zero-shot systems; Qwen3.5-0.8B (60.2, 43.6) trades validity for calls. Three models overlap at (0,0) with 0% validity and are jittered for legibility (values in parentheses are JSON-valid rates). Star: Qwen2.5-0.5B+LoRA adaptor (100, 100), supervised in-distribution reference evaluated on official test ($n = 1{,}934$).

Table 5. Official speaker-disjoint test results (%, $n = 1{,}934$).

System	Parse	Valid	Exact	R_{call}^{exact}	Template exact
Always abstain	100.00	100.00	50.00	0.00	75.71
Lexical control	100.00	100.00	**77.92**	**55.84**	**89.07**
Qwen2.5-0.5B	59.15	0.00	0.00	0.00	0.00
Qwen3.5-0.8B	100.00	73.47	54.65	53.98	56.28
TinyLlama-1.1B	53.67	0.00	0.00	0.00	0.00
SmolLM2-1.7B	93.64	0.78	0.00	0.00	0.00
Qwen3.5-2B	100.00	100.00	50.98	1.96	76.11

abstain-only). Meanwhile, Qwen3.5-2B goes from 50.98% on official (with 19 correct calls, $R_{call}^{exact} = 1.96\%$) to 50.00% on phrase-disjoint (0 calls), and the lexical control is stable (77.92% $\rightarrow$ 77.40%). The official split can therefore overestimate generalizability for models sensitive to template overlap.

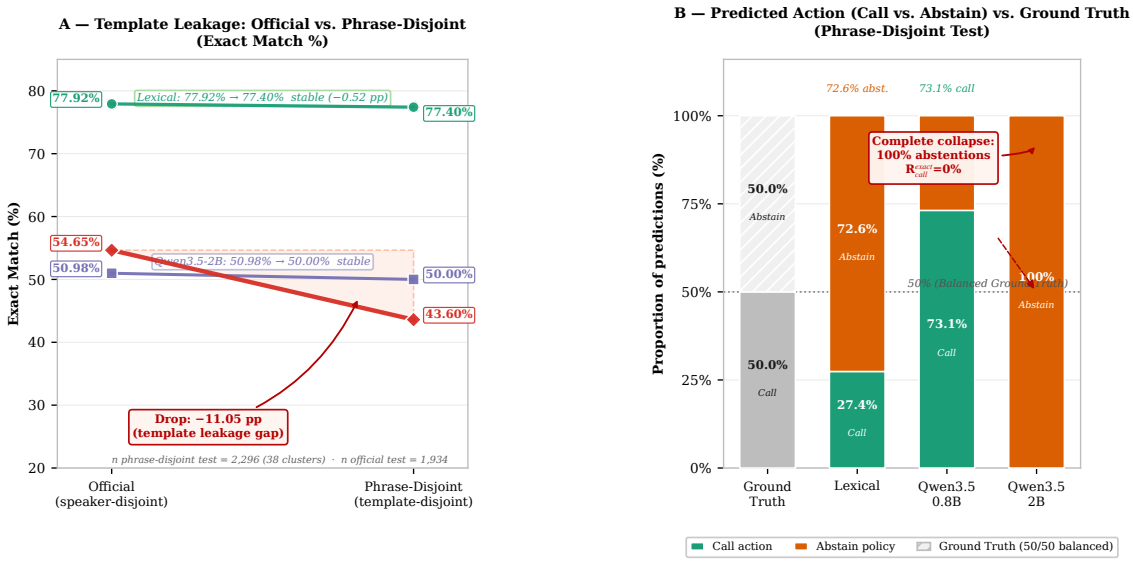

Figure 2. Template leakage and policy collapse (phrase-disjoint test is 50% call / 50% abstain). A Exact match on official (speaker-disjoint, $n = 1{,}934$) vs. phrase-disjoint ($n = 2{,}296$, 38 template clusters). Lexical: 77.92% $\rightarrow$ 77.40% (-0.52 pp, stable); Qwen3.5-2B: 50.98% $\rightarrow$ 50.00% (stable, degenerate abstention); Qwen3.5-0.8B: 54.65% $\rightarrow$ 43.60% (-11.05 pp, shaded gap) with template-cluster 95% CI [26.02, 63.35] on phrase-disjoint. B Predicted action distribution vs. balanced ground truth. Lexical: 27.4% call / 72.6% abstain; Qwen3.5-0.8B: 73.1% call / 26.9% abstain; Qwen3.5-2B: 0% call / 100% abstain (complete collapse, exact call recall 0).

6.3. Structural Error Audit and Latency

The failure modes are qualitatively different. Qwen2.5-0.5B frequently emits JSON-like objects with an operation name in `action`, rather than the required `action=call` plus a tool field. TinyLlama frequently explains the answer or reaches the 64-token cap

before closing the object. SmolLM2 often emits a plausible JSON object with the wrong field structure. These cases explain why parseability is not interchangeable with contract validity.

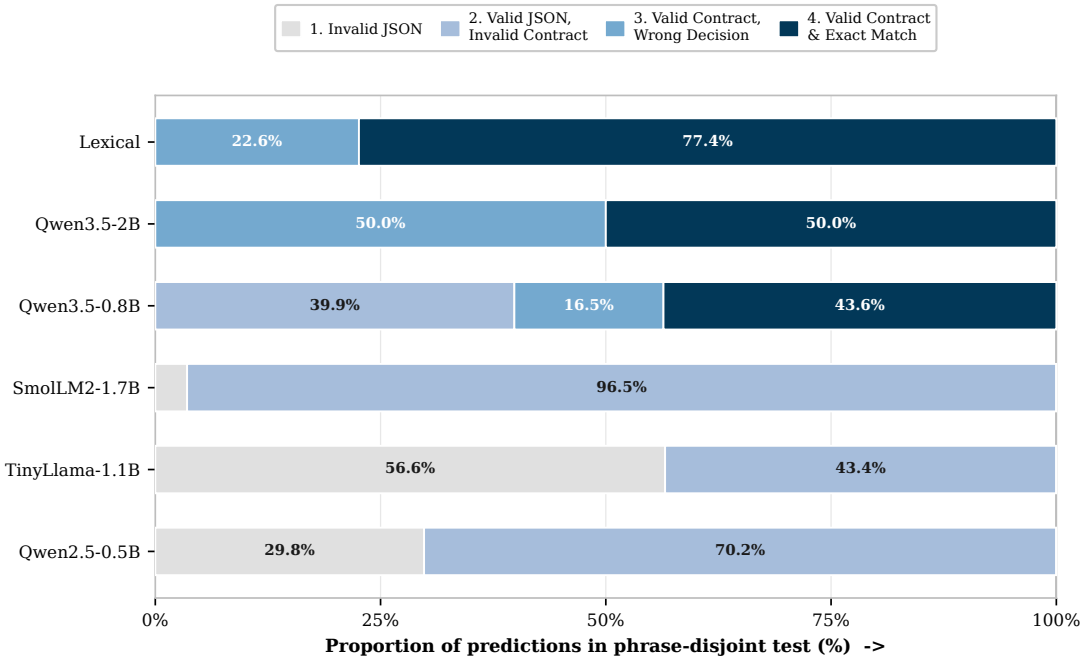

Figure 3. Structural cascade on the phrase-disjoint test ($n = 2{,}296$). Each bar is 100% of a system's predictions decomposed into four mutually exclusive stages: (1) invalid JSON, (2) valid JSON but contract-invalid, (3) contract-valid but wrong decision, (4) contract-valid and exact. Lexical and Qwen3.5-2B have no structural failure (stages 1–2 $= 0$); Qwen3.5-0.8B loses 39.85 pp at stage 2, while Qwen2.5-0.5B, TinyLlama-1.1B and SmolLM2-1.7B obtain 0% exact matches because all parseable output falls in stage 2. Labels shown only for slices $\geq 8\%$.

Server-side single-request latency is reported only as an engineering observation. On phrase-disjoint, mean / median / p95 milliseconds were: Qwen2.5-0.5B 81.4 / 91.0 / 176.0; Qwen3.5-0.8B 324.1 / 327.2 / 448.9; TinyLlama 301.5 / 300.2 / 311.6; SmolLM2 122.9 / 106.7 / 220.1; and Qwen3.5-2B 345.1 / 338.4 / 395.9. These values depend on server load, Transformers implementation, prompt length, and decoding behavior; they are not smartphone measurements.

7. Discussion

The benchmark does not support a monotonic conclusion that larger scale implies better performance. The 2.274B Qwen3.5 checkpoint is structurally compliant but conservative to the point of abstaining on calls. The 0.873B Qwen3.5 checkpoint routes more calls but loses both contract validity and exactness on unseen templates. The external-family checkpoints demonstrate that instruction tuning and model size alone do not guarantee compatibility with a strict output contract.

The lexical control is essential. Without it, the 50% balanced abstention baseline could make a model that refuses everything appear competitive. Cluster analysis is also essential: repeated templates give some systems much larger effective row counts than the number of independent phrasings. The phrase-disjoint test has only 38 clusters, so its uncertainty is broad; nevertheless, it answers a different question from speaker transfer and should not be merged with the official result.

The derived abstention labels require particular caution. They express the declared benchmark policy, not human judgments that a command is unsafe or unsupported in a production assistant. In the framing of selective classification [15], our evaluation assesses policy-defined contract rejection under a fixed schema rather than dynamic risk-coverage optimization based on output logits. A deployed system would require an independent policy layer, schema validator, confidence calibration, and execution authorization. The present experiment intentionally stops before all of those layers.

In contrast, two techniques would change the reading of Figures 1–3 but are deliberately kept outside the zero-shot matrix. As a proof of concept, the retained Qwen2.5-0.5B+LoRA adaptor [10] (star in Figure 1) reaches 100% contract validity and 100% exact match on the official test split ($n = 1{,}934$); as an in-distribution supervised adaptor, it demonstrates that the contract is readily learnable with parameter-efficient fine-tuning, not that unconstrained zero-shot routing is solved, and is therefore not commensurable with the greedy zero-shot results. Constrained decoding via grammar-enforced generation (GBNF grammars in `llama.cpp`, Outlines [11], SynCode [12]) would likewise guarantee 100% validity by construction and collapse stages 1–2 of Figure 3 to zero. This enforces syntactic compliance without implying semantic correctness: a grammar can force a valid `{action, tool, arguments}` shape while leaving call-vs-abstain discrimination and argument exactness to the model. We therefore keep the main benchmark unconstrained and report validity separately from exact match, treating LoRA and grammar-constrained decoding as explicit, non-zero-shot baselines for future work.

8. Reproducibility and Data Governance

The companion benchmark repository records the model matrix, checkpoint revisions, parameter-count method, generic contract, deterministic FSC generator, both split manifests, validator, controls, common inference script, cluster-aware evaluator, runner, logs, and aggregate reports. From the repository root, the reproducible main command (offline, reusing cached predictions) is:

```
HF_HUB_OFFLINE=1 python scripts/run_ultrasmall_benchmark.py \
  --skip-training --skip-existing
```

If an isolated interpreter is required, prefix with the local environment, e.g., `.venv/bin/python scripts/run_ultrasmall_benchmark.py`.

The raw FSC archive and derived JSONL files remain server-local. The source license is recorded as CC BY-NC-ND 4.0 for academic research; no audio, transcript, or per-example prediction is redistributed in Git. Aggregate metrics are sufficient to audit the reported claims without publishing restricted data.

9. Limitations and Next Gates

1. FSC is an English smart-home/assistant corpus, not a native Android or Brazilian-Portuguese command corpus.
2. The four call labels and all abstention labels are derived by a manually declared bridge rather than a native multi-turn dialog context.
3. The task consumes transcriptions, so ASR errors and acoustic variability are not evaluated.
4. The phrase-disjoint test has only 38 unique template clusters, leading to wide bootstrap uncertainty intervals.
5. Native chat templates differ across model families; this is logged but remains a possible prompt-format confound.
6. The main matrix is zero-shot and uses one deterministic decoding setting and one checkpoint revision per model; it does not estimate training variance.
7. No model output is executed, and no result is measured on the physical phone.
8. The benchmark evaluates a compact two-tool contract; evaluating richer task-oriented spoken datasets such as SLURP [2] and STOP [3], as well as standard API tool leaderboards like BFCL [14], constitutes a natural cross-domain extension.

10. Conclusion

We provide a leakage-aware, reproducible benchmark for five ultra-small language models on structured routing from human-origin speech transcriptions. The central result is negative but useful: under a strict canonical contract, a transparent lexical control outperforms every zero-shot model, while the largest model's perfect contract validity is mostly universal abstention. The benchmark therefore separates output compliance, policy abstention, and semantic exactness, and makes phrase leakage visible. It is a defensible first article for the ultra-small-model benchmark scope; Android deployment, ASR, Portuguese human data, and physical-device evaluation are future studies rather than claims of this paper.

References

[1] L. Lugosch, M. Ravanelli, P. Ignoto, V. Naumovich, and Y. Bengio. Speech model pre-training for end-to-end spoken language understanding. In *Interspeech*, 2019.

[2] E. Bastianelli, A. Vanzo, P. Swietojanski, and V. Rieser. SLURP: A Spoken Language Understanding Resource Package. In *Proceedings of EMNLP*, 2020.

[3] P. Tomasello, A. Radfar, S. Raju, D. Gaur, et al. STOP: A dataset for spoken task-oriented semantic parsing. In *IEEE Spoken Language Technology Workshop (SLT)*, 2023.

[4] A. Coucke, A. Saade, A. Ball, T. Bluche, et al. Snips Voice Platform: an embedded Spoken Language Understanding system for private-by-design voice interfaces. *arXiv:1805.10190*, 2018.

[5] Z. Liu, C. Zhao, I. Feldman, et al. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. In *International Conference on Machine Learning (ICML)*, 2024.

[6] M. Abdin, S. A. Jyothi, C. Aneja, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv:2404.14219*, 2024.

[7] P. Zhang, S. Zeng, T. Du, et al. TinyLlama: An open-source small language model. *arXiv:2401.02385*, 2024.

[8] L. Lozhkov, L. Tunstall, A. Mustafa, et al. SmolLM2: When Smol Goes Big—Data-centric training of a small language model. *arXiv:2502.02737*, 2025.

[9] Qwen Team. Qwen3.5 model card. *Hugging Face*, 2026. `https://huggingface.co/Qwen/Qwen3.5-2B`.

[10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[11] B. T. Willard and R. Louf. Efficient guided generation for large language models. *arXiv:2307.09702*, 2023.

[12] S. Ugur, A. Shrivastava, and C. Khurana. SynCode: LLM generation with grammar augmentation. *Transactions on Machine Learning Research (TMLR)*, 2025.

[13] T. Schick, J. Dwivedi-Yu, R. Dessì, et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[14] S. G. Patil, H. Mao, C. Shen, et al. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation. In *International Conference on Machine Learning (ICML)*, 2025.

[15] Y. Geifman and R. El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.